



Spiking neural networks modelled as timed automata: with parameter learning

Elisabetta De Maria¹ · Cinzia Di Giusto¹ · Laetitia Laversa¹

Published online: 17 January 2019
© Springer Nature B.V. 2019

Abstract

In this paper we address the issue of automatically learning parameters of spiking neural networks. Biological neurons are formalized as timed automata and synaptical connections are represented as shared channels among these automata. Such a formalism allows us to take into account several time-related aspects, such as the influence of past inputs in the computation of the potential value of each neuron, or the presence of the refractory period, a lapse of time immediately following the spike emission in which the neuron cannot emit. The proposed model is then formally validated: more precisely, we ensure that some relevant properties expressed as temporal logical formulae hold in the model. Once the validation step is accomplished, we take advantage of the proposed model to write an algorithm for learning synaptical weight values such that an expected behavior can be displayed. The technique we present takes inspiration from supervised learning ones: we compare the effective output of the network to the expected one and backpropagate proper corrective actions in the network. We develop several case studies including a mutual inhibition network.

Keywords Neural networks · Parameter learning · Timed automata · Temporal logic · Model checking

1 Introduction

The brain behaviour is the object of thorough studies: researchers are interested not only in the inner functioning of neurons (which are its elementary components), their interactions and the way these aspects participate to the ability to move, learn or remember, typical of living beings; but also in reproducing such capabilities (emulating nature), e.g., within robot controllers, speech/text/face recognition applications, etc. In order to achieve a detailed understanding of the brain functioning, both neurons behaviour and their interactions must be studied. Several models of the neuron behaviour have been proposed: some of them make neurons behave as binary threshold gates, other ones exploit a sigmoidal transfer function, while, in many cases, differential equations are employed. According to Paugam-Moisys and Bohte (2012) and Maass (1997), three different and progressive *generations* of neural

networks can be recognised: (1) *first generation* models handle discrete inputs and outputs and their computational units are threshold-based transfer functions; they include McCulloch and Pitts' threshold gate model (McCulloch and Pitts 1943), the perceptron model (Freund and Schapire 1999), Hopfield networks (Hopfield 1988), and Boltzmann machines (Ackley et al. 1988); (2) *second generation* models exploit real valued activation functions, e.g., the sigmoid function, accepting and producing real values: a well known example is the multi-layer perceptron (Cybenko 1989; Rumelhart et al. 1988); (3) *third generation* networks are known as spiking neural networks. They extend second generation models treating time-dependent and real valued signals often composed by *spike trains*. Neurons may fire output spikes according to threshold-based rules which take into account input spike magnitudes and occurrence times (Paugam-Moisys and Bohte 2012).

The core of our analysis are *spiking neural network* *s* (Gerstner and Kistler 2002). Because of the introduction of timing aspects they are considered closer to the actual brain functioning than other generations models. Spiking neurons emit spikes taking into account input impulses strength and their occurrence instants. Models of this sort

✉ Cinzia Di Giusto
cinzia.digiusto@gmail.com

¹ Université Côte d'Azur, CNRS, I3S, Sophia Antipolis, France

are of great interest, not only because they are closer to natural neural networks behaviour, but also because the temporal dimension allows to represent information according to various *coding schemes* (Recce 1999; Paugam-Moisy and Bohte 2012): e.g. the amount of spikes occurred within a given time window (*rate coding*), the reception/absence of spikes over different synapses (*binary coding*), the relative order of spikes occurrences (*rate rank coding*), or the precise time difference between any two successive spikes (*timing coding*).

Several spiking neuron models have been proposed in the literature, having different complexities and capabilities. In Izhikevich (2004), Izhikevich classifies spiking neuron models according to some *behaviour* (i.e., typical responses to an input pattern) that they should exhibit in order to be considered biologically relevant. The leaky integrate and fire (LI&F) model (Lapicque 1907), where past inputs relevance exponentially decays with time, is one of the most studied neuron models because it is straightforward and easy to use (Izhikevich 2004; Paugam-Moisy and Bohte 2012). On the other end of the spectrum, the Hodgkin–Huxley (H–H) model (Hodgkin and Huxley 1952) is one of the most complex being composed by four differential equations comparing neurons to electrical circuits. In Izhikevich (2004), the H–H model can reproduce all behaviours under consideration, but the simulation process is really expensive even for just a few neurons being simulated for a small amount of time. Our aim is to produce a neuron model being meaningful from a biological point of view but also amenable to formal analysis and verification, that could be therefore used to detect non-active portions within some network (i.e., the subset of neurons not contributing to the network outcome), to test whether a particular output sequence can be produced or not, to prove that a network may never be able to emit, to assess if a change to the network structure can alter its behaviour, or to investigate (new) learning algorithms which take time into account.

In this paper we focus on the *leaky integrate and fire* (LI&F) model originally proposed in Lapicque (1907). It is a computationally efficient approximation of single-compartment model (Izhikevich 2004) and is abstracted enough to be able to apply formal verification techniques such as model-checking. Here we work on an extended version of the discretised formulation proposed in De Maria et al. (2016), which relies on the notion of logical time. Time is considered as a sequence of logical discrete instants, and an instant is a point in time where external input events can be observed, computations can be done, and outputs can be emitted. The variant we introduce here takes into account some new time-related aspects, such as a lapse of time in which the neuron is not active, i.e., it cannot receive and emit. We encode LI&F networks into timed automata: we

show how to define the behaviour of a single neuron and how to build a network of neurons. Timed automata (Alur and Dill 1994) are finite state automata extended with timed behaviours: constraints are allowed to limit the amount of time an automaton can remain within a particular state, or the time interval during which a particular transition may be enabled. Timed automata networks are sets of automata that can synchronise over *channel* communications.

Our modelling of spiking neural networks consists of timed automata networks where each neuron is an automaton. Its behaviour consists in accumulating the weighted sum of inputs, provided by a number of ingoing weighted synapses, for a given amount of time. Then, if the *potential* accumulated during the last and previous accumulation periods overcomes a given threshold, the neuron fires an output over the outgoing synapse. Synapses are channels shared between the timed automata representing neurons, while *spike* emissions are represented by *broadcast synchronisations* occurring over such channels. Timed automata are also exploited to produce or *recognise* precisely defined spike sequences.

As a first main contribution, we analyse some intrinsic properties of the proposed model, e.g., the maximum threshold value allowing a neuron to emit, or the lack of inter-spike memory, preventing the behaviour of a neuron from being influenced by what happened before the last spike. Furthermore, we encode in temporal logics all the behaviours (or capabilities) a LI&F model should be able to reproduce according to Izhikevich and we exploit model checking to prove these behaviours are reproducible in our model. Izhikevich also identifies a set of behaviours which are not expected to be reproducible by any LI&F model. We prove these limits to hold for our model, too, and we provide, for each non-reproducible behaviour, an extension of the model allowing to reproduce it.

As a second main contribution, we exploit our automata-based modelling to propose a new methodology for parameter inference in spiking neural networks. In particular, our approach allows to find an assignment for the synaptical weights of a given neural network such that it can reproduce a given behaviour. We apply the proposed approach to find suitable parameters in mutual inhibition networks, a well studied class of networks in which the constituent neurons inhibit each other neuron's activity (Matsuoka 1987).

In this work we do not intend to treat classical classification problems of artificial intelligence, but we focus on biological neural networks and aim at studying the behavior of small circuits. The basic biological hypothesis is that neurons in our brain tend to form some mini-circuits with a relevant structure and behavior, which are often referred as archetypes (De Maria et al. 2016). These small

circuits (e.g., simple series, parallel composition, negative loop, inhibition of a behavior, etc.) can be seen as the constituting bricks of bigger networks. We are interested in finding parameters such that these small graphs behave as expected (e.g., for suitable parameters a negative loop exhibits an oscillating trend), not to compete with (simulation) approaches dealing with big networks.

This paper is an extended and revised version of the conference papers (De Maria and Di Giusto 2018; De Maria et al. 2017a). In particular, in Sect. 5 we propose a refined version of the Advice Back-Propagation algorithm. Furthermore, we add a second learning technique that is based on simulation instead of model checking. We also show that the two techniques can be combined.

The rest of the paper is organised as follows: in Sect. 2 we recall definitions of timed automata network s, temporal logics, and model checking; in Sect. 3 we describe our reference model, the LI&F one, and its encoding into timed automata network s; in Sect. 4 we study some intrinsic properties of the obtained model and we validate it against its ability of reproducing or not some behaviours; in Sect. 5 we develop the novel parameter learning approach and we introduce several case studies; in Sect. 6 we give an overview of the related work. Finally, Sect. 7 summarises our contribution and presents some future research directions.

2 Preliminaries

In this section we introduce the formal tools we adopt in the rest of the paper, namely timed automata and temporal logics.

2.1 Timed automata

Timed automata (Alur and Dill 1994) are a powerful theoretical formalism for modelling and verifying real time systems. A timed automaton is an annotated directed (and connected) graph, with an initial node and provided with a finite set of non-negative real variables called *clocks*. Nodes (called *locations*) are annotated with *invariants* (predicates allowing to enter or stay in a location), arcs with *guards*, *communication labels*, and possibly with some variables upgrades and clock *resets*. Guards are conjunctions of elementary predicates of the form $xopc$, where $op \in \{>, \geq, =, <, \leq\}$, x is a clock, and c a (possibly parameterised) positive integer constant. As usual, the empty conjunction is interpreted as true. The set of all guards and invariant predicates will be denoted by G .

Definition 1 A *timed automaton* TA is a tuple $(L, l^0, X, \Sigma, Arcs, Inv)$, where

- L is a set of locations with $l^0 \in L$ the initial one
- X is the set of clocks,
- Σ is a set of communication labels,
- $Arcs \subseteq L \times (G \cup \Sigma \cup U) \times L$ is a set of arcs between locations with a guard in G , a communication label in $\Sigma \cup \{\varepsilon\}$, and a set of variable upgrades (e.g., clock resets);
- $Inv : L \rightarrow G$ assigns invariants to locations.

It is possible to define a synchronised product of a set of timed automata that work and synchronise in parallel. The automata are required to have disjoint sets of locations, but may share clocks and communication labels which are used for synchronisation. We restrict communications to be *broadcast* through labels $b!, b? \in \Sigma$, meaning that a set of automata can synchronise if one is emitting; notice that a process can always emit (e.g., $b!$) and the receivers ($b?$) must synchronise if they can.

Locations can be normal, urgent or committed. Urgent locations force the time to freeze, committed ones freeze time and the automaton must leave the location as soon as possible, i.e., they have higher priority.

The synchronous product $TA_1 \parallel \dots \parallel TA_n$ of timed automata, where $TA_j = (L_j, l_j^0, X_j, \Sigma_j, Arcs_j, Inv_j)$ and L_j are pairwise disjoint sets of locations for each $j \in [1, \dots, n]$, is the timed automaton

$$TA = (L, l^0, X, \Sigma, Arcs, Inv)$$

such that:

- $L = L_1 \times \dots \times L_n$ and $l^0 = (l_1^0, \dots, l_n^0)$, $X = \bigcup_{j=1}^n X_j$, $\Sigma = \bigcup_{j=1}^n \Sigma_j$,
- $\forall l = (l_1, \dots, l_n) \in L : Inv(l) = \bigwedge_j Inv_j(l_j)$,
- $Arcs$ is the set of arcs $(l_1, \dots, l_n) \xrightarrow{g, a, r} (l'_1, \dots, l'_n)$ such that for all $1 \leq j \leq n$ then $l'_j = l_j$.

Its semantics is the one of the underlying timed automaton TA with the following notations. A location is a vector $l = (l_1, \dots, l_n)$. We write $l[l'_j/l_j, j \in S]$ to denote the location l in which the j th element l_j is replaced by l'_j , for all j in some set S . A valuation is a function v from the set of clocks to the non-negative reals. Let $\mathbb{V}$ be the set of all clock valuations, and $v_0(x) = 0$ for all $x \in X$. We shall denote by $v \models F$ the fact that the valuation v satisfies (makes true) the formula F . If r is a clock reset, we shall denote by $v[r]$ the valuation obtained after applying the clock reset $r \subseteq X$ to v ; and if $d \in \mathbb{R}_{>0}$ is a delay, $v + d$ is the valuation such that, for any clock $x \in X$, $(v + d)(x) = v(x) + d$.

The semantics of a synchronous product $TA_1 \parallel \dots \parallel TA_n$ is defined as a timed transition system $(S, s_0, \rightarrow)$, where $S = (L_1 \times \dots \times L_n) \times \mathbb{V}$ is the set of states, $s_0 = (l^0, v_0)$ is the initial state, and $\rightarrow \subseteq S \times S$ is the transition relation defined by:

- (silent): $(l, v) \rightarrow (l', v')$ if there exists $l_i \xrightarrow{g_i, e, r} l'_i$, for some i , such that $l' = l[l'_i/l_i]$, $v \models g$ and $v' = v[r]$,
- (broadcast): $(\bar{l}, v) \rightarrow (\bar{l}', v')$ if there exists an output arc $l_j \xrightarrow{g_j, b!, r_j} l'_j \in \text{Arcs}_j$ and a (possibly empty) set of input arcs of the form $l_k \xrightarrow{g_k, b?, r_k} l'_k \in \text{Arcs}_k$ such that for all $k \in K = \{k_1, \dots, k_m\} \subseteq \{l_1, \dots, l_n\} \setminus \{l_j\}$, the size of K is maximal, $v \models \bigwedge_{k \in K \cup \{j\}} g_k$, $l' = l[l'_k/l_k, k \in K \cup \{j\}]$ and $v' = v[r_k, k \in K \cup \{j\}]$;
- (timed): $(l, v) \rightarrow (l, v + d)$ if $v + d \models \text{Inv}(l)$.

The valuation function v is extended to handle a set of shared bounded integer variables: predicates concerning such variables can be part of edges guards or locations invariants, moreover variables can be updated on edges firings but they cannot be assigned to or from clocks.

Example 1 In Fig. 1 we consider the network of timed automata TA_1 and TA_2 with broadcast communications, and we give a possible run. TA_1 and TA_2 start in the l_1 and l_3 locations, respectively, so the initial state is $[(l_1, l_3); x = 0]$. A *timed* transition produces a delay of 1 time unit, making the system move to state $[(l_1, l_3); x = 1]$. A *broadcast* transition is now enabled, making the system move to state $[(l_2, l_3); x = 0]$, broadcasting over channel a and resetting the x clock. Two successive *timed* transitions (0.5 time units) followed by a *broadcast* one will eventually lead the system to state $[(l_2, l_4); x = 1]$.

Throughout our modelling, we have used the specification and analysis tool Uppaal (Bengtsson et al. 1995), which provides the possibility of designing and simulating timed automata network s on top of the ability of testing networks against temporal logic formulae. All figures depicting timed automata follow the graphic conventions of

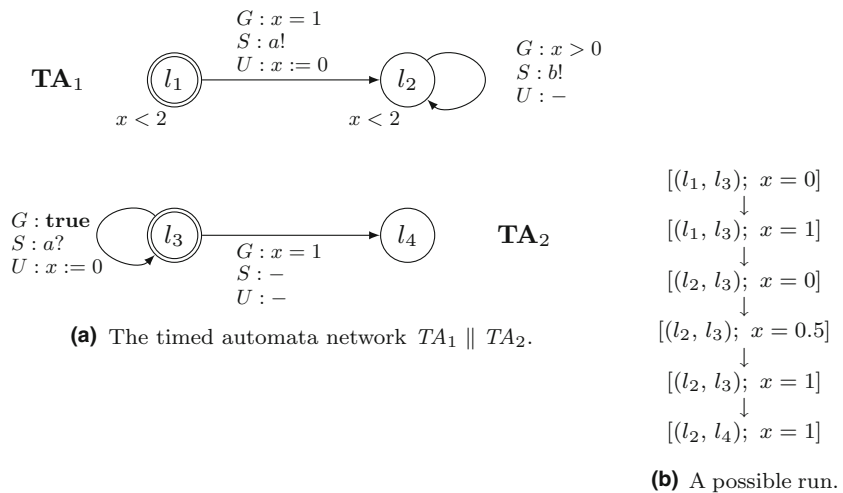
the tool (e.g., initial states are denoted with a double circle).

2.2 Temporal logics and model checking

Model checking is one of the most common approaches to the verification of software and hardware (distributed) systems (Clarke et al. 1999). It allows to automatically prove whether a system verifies or not a given specification. In order to apply such a technique, the system at issue should be encoded as a finite transition system and the specification should be written using propositional temporal logic. Formally, a transition system over a set AP of atomic propositions is a tuple $M = (Q, T, L)$, where Q is a finite set of states, $T \subseteq Q \times Q$ is a total transition relation, and $L : Q \rightarrow 2^{AP}$ is a labelling function that maps every state into the set of atomic propositions that hold at that state.

Temporal formulae describe the dynamical evolution of a given system. The computation tree logic CTL^* allows to describe properties of computation trees. Its formulas are obtained by (repeatedly) applying boolean connectives ($\wedge, \vee, \neg, \rightarrow$), *path quantifiers*, and *state quantifiers* to atomic formulas. The path quantifier **A** (resp., **E**) can be used to state that all the paths (resp., some path) starting from a given state have some property. The state quantifiers are **X** (next time), which specifies that a property holds at the next state of a path, **F** (sometimes in the future), which requires a property to hold at some state on the path, **G** (always in the future), which imposes that a property is true at every state on the path, and **U** (until), which holds if there is a state on the path where the second of its argument properties holds and, at every preceding state on the path, the first of its two argument properties holds. Given two formulas φ_1 and φ_2 , in the rest of the paper we use the shortcut $\varphi_1 \rightsquigarrow \varphi_2$ to denote the liveness property

Fig. 1 A network of timed automata with a possible run



$AG(\varphi_1 \rightarrow AF\varphi_2)$, which can be read as “ φ_1 always leads to φ_2 ”.

The branching time logic CTL is a fragment of CTL* that allows quantification over the paths starting from a given state. Unlike CTL*, it constrains every state quantifier to be immediately preceded by a path quantifier.

Given a transition system $M = (Q, T, L)$, a state $q \in Q$, and a temporal logic formula φ expressing some desirable property of the system, the *model checking problem* consists of establishing whether φ holds at q or not, namely, whether $M, q \models \varphi$.

3 Leaky integrate and fire model and mapping to timed automata

Spiking neural networks (Maass 1997) are modelled as directed weighted graphs where vertices are computational units and edges represent *synapses*. The signals propagating over synapses are trains of impulses: *spikes*. Synapses may modulate these signals according to their weight: *excitatory* if positive, or *inhibitory* if negative.

The dynamics of neurons is governed by their *membrane potential* (or, simply, *potential*), representing the difference of electrical potential across the cell membrane. The membrane potential of each neuron depends on the spikes received over the ingoing synapses. Both current and past spikes are taken into account, even if old spikes contribution is lower. In particular, the *leak factor* is a measure of the neuron memory about past spikes. The neuron outcome is controlled by the algebraic difference between its membrane potential and its *firing threshold*: it is enabled to fire (i.e., emit an output impulse over *all* outgoing synapses) only if such a difference is non-negative. Spike propagation is assumed to be instantaneous. Immediately after each emission the neuron membrane potential is reset and the neuron stays in a *refractory period* for a given amount of time. During this period it has no dynamics: it cannot increase its potential as any received spike is lost and therefore it cannot emit any spike.

Definition 2 (*Spiking integrate and fire neural network*) A *spiking integrate and fire neural network* is a tuple (V, A, w) , where:

- V are spiking integrate and fire neurons,
- $A \subseteq V \times V$ are synapses,
- $w : A \rightarrow \mathbb{Q} \cap [-1, 1]$ is the synapse weight function associating to each synapse (u, v) a weight $w_{u,v}$.

We distinguish three disjoint sets of neurons: V_i (input neurons), V_{int} (intermediary neurons), and V_o (output neurons), with $V = V_i \cup V_{int} \cup V_o$.

A *spiking integrate and fire neuron* v is characterized by a parameter tuple

$$(\theta_v, \tau_v, \lambda_v, p_v, y_v),$$

where

- $\theta_v \in \mathbb{N}$ is the *firing threshold*,
- $\tau_v \in \mathbb{N}^+$ is the *refractory period*,
- $\lambda_v \in \mathbb{Q} \cap [0, 1]$ is the *leak factor*.

The dynamics of a *spiking integrate and fire neuron* v is given by:

- $p_v : \mathbb{N} \rightarrow \mathbb{Q}_0^+$ is the [membrane] *potential* function defined as

$$p_v(t) = \begin{cases} \sum_{i=1}^m w_i \cdot x_i(t), & \text{if } p_v(t-1) \geq \theta_v \\ \sum_{i=1}^m w_i \cdot x_i(t) + \lambda_v \cdot p_v(t-1), & \text{otherwise.} \end{cases}$$

with $p_v(0) = 0$ and where $x_i(t) \in \{0, 1\}$ is the signal received at the time t by the neuron through its i th out of m input synapses (observe that the past potential is multiplied by the leak factor while current inputs are not weakened),

- $y_v : \mathbb{N} \rightarrow \{0, 1\}$ is the neuron output function, defined as

$$y_v(t) = \begin{cases} 1 & \text{if } p_v(t) \geq \theta_v \\ 0 & \text{otherwise.} \end{cases}$$

As shown in the previous definition, the set of neurons of a spiking integrate and fire neural network can be classified into input, intermediary, and output ones. Each input neuron can only receive as input external signals (and not other neurons' output). The output of each output neuron is considered as an output for the network. Output neurons are the only ones whose output is not connected to other neurons.

We present here our modelling of spiking integrate and fire neural networks (in the following denoted as neural networks) via timed automata network s. Let $S = (V, A, w)$ be a neural network, G be a set of *input generator* neurons (these fictitious neurons are connected to input neurons and generate input sequences for the network), and O be a set of *output consumer* neurons (these fictitious neurons are connected to the broadcast channel of each output neuron and aim at consuming their emitted spikes). The corresponding timed automata network is obtained as the synchronous product of the encoding of input generator neurons, the neurons of the network (referred as standard neurons in the following), and output consumers neurons. More formally:

$$\llbracket S \rrbracket = (\llbracket n_g \rrbracket_{g \in G}) \parallel (\llbracket v_j \rrbracket_{j \in V}) \parallel (\llbracket n_c \rrbracket_{c \in O})$$

3.1 Input generators

The behaviour of input generator neurons is part of the specification of the network. Here we define two kinds of input behaviours: regular and non-deterministic ones. For each family, we provide an encoding into timed automata.

3.1.1 Regular input sequences

Spike trains are “regular” sequences of spikes and pauses: spikes are instantaneous while pauses have a non-null duration. Sequences can be *empty*, *finite* or *infinite*. After each spike there must be a pause, except when the spike is the last event of a finite sequence. Infinite sequences are composed by two parts: a finite and arbitrary prefix and an infinite and periodic part composed by a finite sequence of *spike–pause* pairs which is repeated infinitely often. More formally, such sequences are given in terms of the following grammar:

$$\begin{aligned} G::= & \Phi \cdot (\Phi)^\omega \mid P(d) \cdot \Phi \cdot (\Phi)^\omega \\ \Phi::= & s \cdot P(d) \cdot \Phi \mid \varepsilon \end{aligned}$$

with s representing a spike and $P(d)$ a pause of duration d . It is possible to generate an emitter automaton for any regular input sequence:

Definition 3 (*Input generator*) Let $I \in \mathcal{L}(G)$ be a word over the language generated by IS , then its encoding into timed automata is $\llbracket I \rrbracket = (L(I), \text{first}(I), \{t\}, \{y\}, \text{Arcs}(I), \text{Inv}(I))$. It is inductively defined as follows:

- if $I := \Phi_1 \cdot (\Phi_2)^\omega$
 - $L(I) = L(\Phi_1) \cup L(\Phi_2)$, where $\text{last}(\Phi_2)$ is *urgent*
 - $\text{first}(I) = \text{first}(\Phi_1)$
 - $\text{Arcs}(I) = \text{Arcs}(\Phi_1) \cup \text{Arcs}(\Phi_2) \cup \{(\text{last}(\Phi_1), \text{true}, \varepsilon, \emptyset, \text{first}(\Phi_2)), (\text{last}(\Phi_1), \text{true}, \varepsilon, \emptyset, \text{first}(\Phi_2))\}$
 - $\text{Inv}(I) = \text{Inv}(\Phi_1) \cup \text{Inv}(\Phi_2)$
- if $I := P(d) \cdot \Phi_1 \cdot (\Phi_2)^\omega$
 - $L(I) = \{\mathbf{P}_0\} \cup L(\Phi_1) \cup L(\Phi_2)$, where $\text{last}(\Phi_2)$ is *urgent*
 - $\text{first}(I) = \mathbf{P}_0$
 - $\text{Arcs}(I) = \text{Arcs}(\Phi_1) \cup \text{Arcs}(\Phi_2) \cup \{(\mathbf{P}_0, t \leq d, \{t := 0\}, \text{first}(\Phi_1)), (\text{last}(\Phi_1), \text{true}, \varepsilon, \emptyset, \text{first}(\Phi_2)), (\text{last}(\Phi_1), \text{true}, \varepsilon, \emptyset, \text{first}(\Phi_2))\}$
 - $\text{Inv}(I) = \{\mathbf{P}_0 \mapsto t \leq d\} \cup \text{Inv}(\Phi_1) \cup \text{Inv}(\Phi_2)$
- if $\Phi := \varepsilon$
 - $L(\Phi) = \{\mathbf{E}\}$
 - $\text{first}(\Phi) = \text{last}(\Phi) = \mathbf{E}$

- $\text{Arcs}(\Phi) = \emptyset$
- $\text{Inv}(\Phi) = \emptyset$
- if $\Phi := s \cdot P(d) \cdot \Phi'$
 - $L(\Phi) = \{\mathbf{S}, \mathbf{P}\} \cup L(\Phi')$
 - $\text{first}(\Phi) = \mathbf{S}$, $\text{last}(\Phi) = \text{last}(\Phi')$
 - $\text{Arcs}(\Phi) = \text{Arcs}(\Phi') \cup \{(\mathbf{S}, \text{true}, y!, \emptyset, \mathbf{P}), (\mathbf{P}, t = d, \varepsilon, \{t := 0\}, \text{first}(\Phi'))\}$
 - $\text{Inv}(\Phi) = \{\mathbf{P} \mapsto t \leq d\} \cup \text{Inv}(\Phi')$

Figure 2 depicts the shape of input generators. Figure 2a shows the generator $\llbracket I \rrbracket$, obtained from $I := \Phi_1 \cdot (\Phi_2)^\omega$. The edge connecting the last state of $\llbracket \Phi_2 \rrbracket$ to the first one allows Φ_2 to be repeated infinitely often. Figure 2b shows the case of an input sequence $I := P(d) \cdot \Phi_1 \cdot (\Phi_2)^\omega$ beginning with a pause $P(d)$: in this case, the initial location of $\llbracket I \rrbracket$ is $\mathbf{P}_0$, which imposes a delay of d time units. The remainder of the input sequence is encoded as for the previous case. Figure 2c shows the induction basis for encoding a sequence Φ , i.e., the case $\Phi := \varepsilon$. It is encoded as a location $\mathbf{E}$ having no edge. Finally, Fig. 2d shows the case of a non-empty spike–pause pair sequence $\Phi := s \cdot P(d) \cdot \Phi'$. It consists of an *urgent* location $\mathbf{S}$: when the automaton moves from $\mathbf{S}$, a spike is fired over channel y and the automaton moves to location $\mathbf{P}$, representing a silent period. After that, the automaton proceeds with the encoding of Φ' .

3.1.2 Non-deterministic input sequences

This kind of input sequences is useful when no assumption is available on neuron inputs. These are random sequences of spikes separated by at least $T_{\min}$ time units.

Such sequences can be generated by an automaton defined as follows:

Definition 4 (*Non-deterministic input generator*) A non-deterministic input generator I_{nd} is a tuple

$$(L, \mathbf{B}, X, \Sigma, \text{Arcs}, \text{Inv}),$$

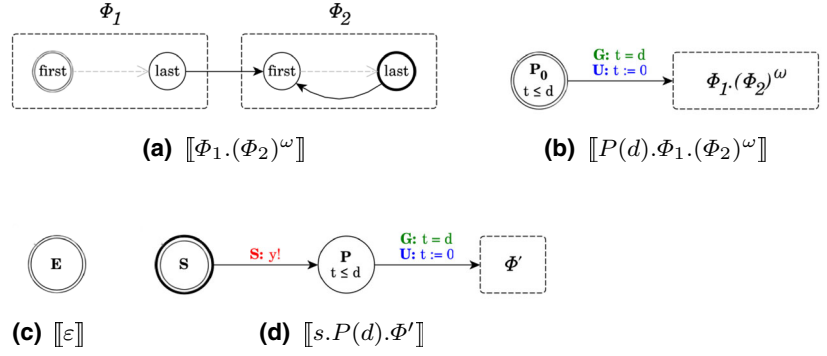
with

- $L = \{\mathbf{B}, \mathbf{S}, \mathbf{W}\}$, with $\mathbf{S}$ *urgent*,
- $X = \{t\}$
- $\Sigma = x$
- $\text{Arcs} = \{(\mathbf{B}, t = D, x!, \emptyset, \mathbf{S}), (\mathbf{S}, \text{true}, \varepsilon, \{t := 0\}, \mathbf{W}), (\mathbf{W}, t > T_{\min}, x!, \emptyset, \mathbf{S})\}$
- $\text{Inv}(\mathbf{B}) = (t \leq D)$

where D is the initial delay.

The behavior of such a generator depends on clock t and broadcast channel x , and can be summarized as follows: it waits in location $\mathbf{B}$ an arbitrary amount of time before moving to location $\mathbf{S}$, firing its first spike over channel

Fig. 2 Representation of the encoding of an input sequence



x . Since location **S** is *urgent*, the automaton instantaneously moves to location **W**, resetting clock t . Finally, from location **W**, after an arbitrary amount of time t , it moves to location **S**, firing a spike. Notice that an initial delay D may be introduced by adding the invariant $t \leq D$ to the location **B** and the guard $t = D$ on the edge (**B** $\rightarrow$ **S**).

3.2 Standard neurons

The neuron is a computational unit behaving as follows: (1) it accumulates potential whenever it receives input spikes within a given *accumulation period*, (2) if the accumulated potential is greater than the *threshold*, it emits an output spike, (3) it waits during a *refractory period*, and restarts from (1). Observe that the accumulation period is not present in the definition of neuron (Definition 2). It is indeed introduced here to slice time and therefore discretise the decrease of the potential value due to the leak factor. We assume that two input spikes on the same synapse cannot be received within the same accumulation period (i.e., the accumulation period is shorter than the minimum refractory period of the input neurons of the network). Next, we give the encoding of neurons into timed automata.

Definition 5 Given a neuron $v = (\theta, \tau, \lambda, p, y)$ with m input synapses, its encoding into timed automata is $\mathcal{N} = (L, \mathbf{A}, X, \text{Var}, \Sigma, \text{Arcs}, \text{Inv})$ with:

- $L = \{\mathbf{A}, \mathbf{W}, \mathbf{D}\}$ with **D** committed,
- $X = \{t\}$
- $\text{Var} = \{p, a\}$
- $\Sigma = \{x_i \mid i \in [1..m]\} \cup \{y\}$,
- $\text{Arcs} = \{(\mathbf{A}, t \leq T, x_i?, \{a := a + w_i\}, \mathbf{A}) \mid i \in [1..m]\} \cup \{(\mathbf{A}, t = T, \{p := a + \lfloor \lambda p \rfloor\}, \mathbf{D}), (\mathbf{D}, p < \theta, \{a := 0\}, \mathbf{A}), (\mathbf{D}, p \geq \theta, y!, \mathbf{W}), (\mathbf{W}, t = \tau, \{a := 0, t := 0, p := 0\}, \mathbf{A})\}$;

- $\text{Inv}(\mathbf{A}) = t \leq T, \text{Inv}(\mathbf{W}) = t \leq \tau, \text{Inv}(\mathbf{D}) = \text{true}$.

The neuron behavior, described by the automaton in Fig. 3a, depends on the following channels, variables and clocks:

- x_i for $i \in [1..m]$ are the m input channels,
- y is the broadcast channel used to emit the output spike,
- $p \in \mathbb{N}$ is the current potential value, initially set to zero,
- $a \in \mathbb{N}$ is the weighted sum of input spikes occurred within the *current* accumulation period; it equals zero at the beginning of each round.

The behaviour of the automaton modelling neuron v can be summed up as follows:

- the neuron keeps waiting in state **A** (for Accumulation) for input spikes while $t \leq T$ and, whenever it receives a spike on input x_i , it updates a with $a := a + w_i$;
- when $t = T$, the neuron moves to state **D** (for Decision), resetting t and updating p according to the potential function given in Definition 2:

$$p := a + \lfloor \lambda \cdot p \rfloor$$

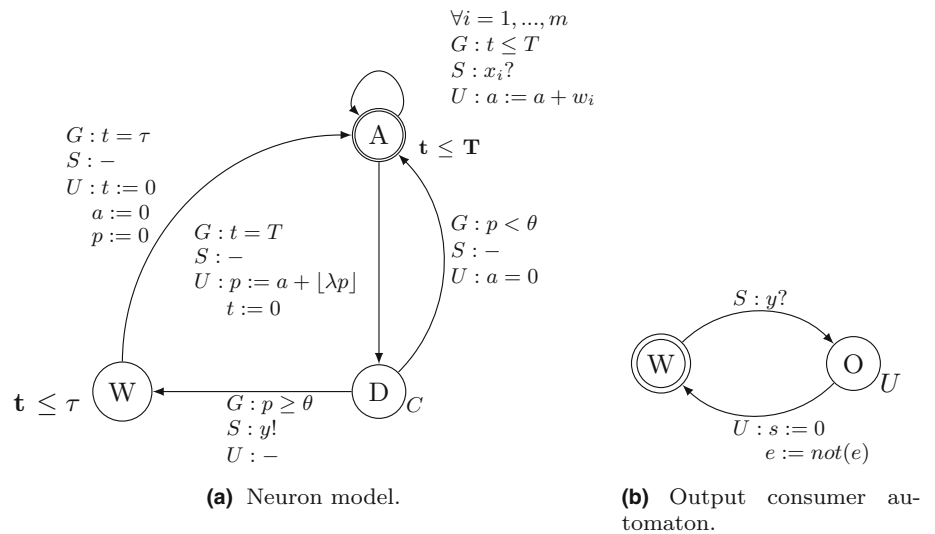
Since state **D** is *committed*, it does not allow time to progress, so, from this state, the neuron can move back to state **A** resetting a if the potential has not reached the threshold $p < \theta$, or it can move to state **W**, firing an output spike, otherwise;

- the neuron remains in state **W** (for Wait) for τ time units (τ is the length of the refractory period) and then it moves back to state **A** resetting a , p and t .

3.3 Output consumers

In order to have a complete modelling of a spiking neural network, for each output neuron we build an *output consumer* automaton O_y . The automaton, whose formal

Fig. 3 Automata for standard neuron and output consumer



definition is straightforward, is shown in Fig. 3b. The consumer waits in location **W** for the corresponding output spikes on channel y and, as soon as it receives the spike, it moves to location **O**. This location is only needed to simplify model checking queries. Since it is urgent, the consumer instantly moves back to location **W** resetting s , the clock measuring the elapsed time since last emission, and setting e to its negation, with e being a *boolean variable* which differentiates each emission from its successor.

Definition 6 (*Output consumer*) An output consumer is a timed automaton

$$\mathcal{N} = (L, \mathbf{W}, X, \text{Var}, \Sigma, \text{Arcs}, \text{Inv})$$

with

- $L = \{\mathbf{W}, \mathbf{O}\}$ with **O** urgent,
- $X = \{s\}$
- $\text{Var} = \{e\}$
- $\Sigma = \{y_i \mid y_i \text{ is an output neuron}\}$
- $\text{Arcs} = \{(\mathbf{W}, y_i?, \mathbf{O}), (\mathbf{O}, s := 0, \{e := \text{not}(e)\}, \mathbf{W})\}$
- $\text{Inv}(\mathbf{W}) = \text{true}, \text{Inv}(\mathbf{O}) = \text{true}.$

We have a complete implementation of the spiking neural network model proposed in the paper via the tool Uppaal. It can be found on the web page (Ciatto et al. 2016). We have validated our neuron model against some characteristic properties studied in Izhikevich (2004) (tonic spiking, excitability, integrator, etc.). These properties have been formalised in temporal logics and checked via model-checking tools.

Observe that, since we rely on a discrete time, we could have used tick automata (Gruber et al. 2005), a variant of Büchi automata where a special clock models the discrete flow of time. However, to the best of our knowledge, no

existing tool allows to implement such automata. We decided to opt for timed automata in order to have an effective implementation of our networks to be exploited in parameter learning algorithms.

4 Validation of the model

In this section we show some properties of the neuron model of Definition 5. The first group of properties are structural. We can compute a minimum value such that any neuron, having a threshold greater than or equal to it, will never be able to fire.

Property 1 Let $\mathcal{N} = (\theta, \tau, \lambda, p, y)$ be a neuron and $a_{\max}$ be the maximum value received during each accumulation period. Then, if $\theta \geq \frac{a_{\max}}{1-\lambda}$, the neuron is not able to fire.

Proof Without loss of generality, we suppose that, during each accumulation period, $\mathcal{N}$ receives the maximum possible input $a_{\max}$. Then, its potential function is:

$$p_n = a_{\max} + \lfloor \lambda \cdot p_{n-1} \rfloor$$

which is always lower than or equal to its undiscretized version:

$$p_n \leq p'_n = a_{\max} + \lambda \cdot p'_{n-1}$$

The same inequality can be written in explicit form:

$$p_n \leq p'_n = \sum_{k=0}^n a_{n-k} \cdot \lambda^k$$

and, since we assumed the neuron always receives $a_{\max}$, a_{n-k} is constant and does not depend on k :

$$p_n \leq a_{\max} \cdot \sum_{k=0}^n \lambda^k$$

The rightmost factor is a geometric series:

$$p_n \leq a_{\max} \cdot \frac{1 - \lambda^{n+1}}{1 - \lambda}$$

which reaches its maximum value $\frac{1}{1-\lambda}$ for $n \rightarrow \infty$, therefore:

$$p_n \leq \frac{a_{\max}}{1 - \lambda}.$$

Thus, if $\theta \geq \frac{a_{\max}}{1-\lambda}$, it is impossible for the neuron potential to reach the threshold and, consequently, the neuron cannot fire. $\square$

In what follows, we only consider neurons that respect the previous constraint.

Apart from the minimum threshold, we can also quantify the amount of time that the neuron requires to complete an accumulate-fire-rest cycle. We show that there exists a minimum delay between neuron emissions.

Property 2 Let $\mathcal{N} = (\theta, \tau, \lambda, p, y)$ be a neuron. Then the time difference between successive firings cannot be lower than $T + \tau$.

Proof Let $A_n = \sum_{k=1}^T a_{k+t_0}$ be the sum of weighted inputs during the n -th accumulation period, then the neuron behaviour can be described as follows:

$$p_n = A_n + \lfloor \lambda \cdot p_{n-1} \rfloor$$

which is the potential value after the n -th accumulation period. If the neuron eventually fires an output spike, then there exists $\hat{n} > 0$ such that:

$$\hat{n} = \arg \min_{n \in \mathbb{N}} \{p_n : p_n \geq \theta\}$$

i.e., the firing will occur at the end of the $\hat{n}$ -th accumulation period, which means during the $\hat{t}$ -th time unit since t_0 , thus:

$$\hat{t} = \hat{n} \cdot T + t_0$$

where t_0 is the last reset time, i.e., the last instant back in time when the neuron completed its refractory period. Then the next reset time t' , i.e., the next instant in future when the neuron will complete its refractory period, after having emitted a spike, is:

$$t' = \hat{t} + \tau = \hat{n} \cdot T + \tau + t_0$$

At instant t' , the neuron quits its refractory period, n is reset to 0, t_0 is set to t' , and $\hat{n}$, $\hat{t}$ and t' must be consequently re-computed as described above.

Such a way to describe our model dynamics allow us to express the *inter-firing period* as a function of $\hat{n}$:

$$t' - t_0 = \hat{n} \cdot T + \tau$$

So, the minimum inter-firing period is $T + \tau$ for $\hat{n} = 1$. $\square$

Such a property can also be verified as follows: let $\mathcal{I}$ be the non-deterministic input generator having $T_{\min} = 1$ and, without loss of generality,¹ let the initial delay $D = T + \tau$. Then the timed automata network $\mathcal{I} \parallel \mathcal{N} \parallel \mathcal{O}$ satisfies the following formula:

$$AG(state_{\mathcal{O}}(\mathbf{O}) \rightarrow eval_{\mathcal{O}}(\mathbf{s}) \geq T + \tau)$$

where s measures the time elapsed since last firing, meaning that, whenever the output consumer receives a spike, the time elapsed since the previous received spike cannot be lower than $T + \tau$.

The next fact states that only positive stimulations are necessary for the neuron to produce emissions.

Fact 1 Let $\mathcal{N} = (\theta, \tau, \lambda, p, y)$ be a neuron, $a(t)$ the sum of weighted inputs received during the current accumulation period, and $p(t-1)$ the neuron potential at the end of the previous accumulation period. If $p(t-1) < \theta$ and $a(t) < 0$, the neuron cannot fire at the end of the current accumulation period. Moreover, if $p(t) \geq \theta$ then $a(t) > 0$.

The neuron potential is affected by every input spike it received since the last reset time, but every event that occurred before that instant is forgotten: i.e., neurons are memoryless.

Definition 7 (Inter-emission memory) Let $\mathcal{N}$ be a neuron, $Z_{\mathcal{N}}$ its reset times set, and I an input sequence. Then $\mathcal{N}$ has inter-emission memory if and only if there exist two different $t, t' \in Z_{\mathcal{N}}$ such that the output sequences produced by $\mathcal{N}$ as a response to I starting from t and t' are different.

Property 3 Neurons have not inter-emission memory.

Proof When the neuron moves from location **W** to **A**, it resets clock t and variables p and a , making them equal to their initial values. This entails that the neuron, if subjected to the same input sequence, will always behave in the same way. $\square$

Next, we validate the neuron model against its ability of reproducing or not some behaviours, as described by Izhikevich (2004). We introduce first three behaviours that are verified by our model.

4.1 Tonic spiking

Tonic spiking is the behaviour of a neuron producing a periodic output sequence as a response to a persistent excitatory constant input sequence.

¹ The initial delay is required in order to make the formula hold for the first output spike too.

Property 4 (Tonic spiking) *Let $\mathcal{N} = (\theta, \tau, \lambda, p, y)$ be a neuron having only one ingoing excitatory synapse of weight w and let $\mathcal{I}$ be the input source connected to $\mathcal{N}$ producing a persistent input sequence. Then $\mathcal{N}$ produces a periodic output sequence.*

The property holds by construction. It can be tested via model checking in the following way. Let $\mathcal{I}$ be the fixed-rate input generator having arbitrary initial delay D , and let $\mathcal{O}$ be an output consumer. Then the timed automata network $\mathcal{I} \parallel \mathcal{N} \parallel \mathcal{O}$ satisfies the following formulae:

$$\left\{ \begin{array}{l} \text{state}_{\mathcal{O}}(\mathbf{O}) \wedge \text{eval}_{\mathcal{O}}(e) \rightsquigarrow \text{state}_{\mathcal{O}}(\mathbf{O}) \wedge \neg \text{eval}_{\mathcal{O}}(e) \\ \text{state}_{\mathcal{O}}(\mathbf{O}) \wedge \neg \text{eval}_{\mathcal{O}}(e) \rightsquigarrow \text{state}_{\mathcal{O}}(\mathbf{O}) \wedge \text{eval}_{\mathcal{O}}(e). \end{array} \right.$$

where $\mathbf{O}$ is the location that the consumer automaton $\mathcal{O}$ reaches after consuming a spike and e is an alternating boolean variable whose value flips whenever $\mathcal{O}$ moves into location $\mathbf{O}$. So, whenever automaton $\mathcal{O}$ reaches location $\mathbf{O}$, it will eventually reach it again.

One may also find the value P of the period of some given neuron $\mathcal{N}$ by means of simulations, thus the periodic behaviour can be proven verifying the following formula:

$$AG(\text{state}_{\mathcal{O}}(\mathbf{O}) \wedge \text{eval}_{\mathcal{N}}(\mathbf{f}) \rightarrow \text{eval}_{\mathcal{O}}(\mathbf{s}) = \mathbf{P})$$

where s is the clock measuring the time elapsed since last spike consumed by $\mathcal{O}$, and f is a boolean variable of automaton $\mathcal{N}$ which is initially *false* and is set to *true* when edge $(\mathbf{W} \rightarrow \mathbf{A})$ fires (i.e., it indicates whether $\mathcal{N}$ has already emitted the first spike and waited the first refractory period or not).

4.2 Integrator

Integrator is the behaviour of a neuron producing an output spike whenever it receives *at least* a specific number of spikes from its input sources in the same accumulation period.

Property 5 (Integrator) *Let $\mathcal{N} = (\theta, \tau, \lambda, p, y)$ be a neuron having m synapses with maximum excitatory weight R and a threshold $n \leq m$. Then the neuron emits if it receives a spike from at least n input sources during the same accumulation period.*

As in the previous case, we can use model checking tools and test the formula stating that, if at least n generators are ready to emit (location $\mathbf{S}$) while $\mathcal{N}$ is in $\mathbf{A}$, then $\mathcal{O}$ will eventually capture an output of $\mathcal{N}$:

$$\left(\sum_{i=1}^m \text{state}_i(\mathbf{S}) \geq n \right) \wedge \text{state}_{\mathcal{N}}(\mathbf{A}) \rightsquigarrow \text{state}_{\mathcal{O}}(\mathbf{O})$$

Notice that, since potential depends on past inputs too, the neuron may still be able to fire in other circumstances, e.g., if it keeps receiving less than n spikes for a sufficient

number of accumulation periods, then it may eventually fire.

4.3 Excitability

Excitability is the behaviour of a neuron emitting sequences having a *decreasing* inter-firing period, i.e., an increasing output frequency, when stimulated by an *increasing* number of excitatory inputs.

Property 6 (Excitability) *Let $\mathcal{N} = (\theta, \tau, \lambda, p, y)$ be a neuron having m excitatory synapses. Then the inter-spike period decreases as the sum of weighted input spikes increases.*

Proof If we assume the neuron is receiving an increasing number of excitatory spikes, generated by an increasing number of input sources emitting persistent inputs, then a_t is the non-negative, non-decreasing and progressing (i.e., $\forall u \exists t : a_t > u$) succession representing the weighted sum of inputs within the t -th time unit. Consequently, $A_n = \sum_{k=1}^T a_{k+t_0}$ is the non-negative, non-decreasing and progressing succession counting the total sum of inputs within the n -th accumulation period. Since A_n is positive and Property 2 holds, we can prove that the inter-spike period $t_n - t_{n-1}$ decreases. $\square$

The following behaviours are not satisfied by the LI&F model, we show that our encoding cannot verify them as well.

4.4 Phasic spiking

Phasic spiking is the behaviour of a neuron producing a *single* output spike when receiving a persistent and excitatory input sequence and then remaining quiescent for the rest of it. Such a behaviour depends on the neuron to have inter-emission memory.

Property 7 *Neurons cannot reproduce the phasic spiking behaviour.*

Proof The phasic spiking behaviour requires the neuron to ignore any excitatory input spike occurring after its first emission. This means producing different outcomes, before and after the first emission, as a response to the same input sequence, which is impossible for a memoryless neuron, as stated in Property 3. $\square$

We can extend our model to reproduce phasic spiking, see Fig. 4. This variant makes the neuron able to “remember” if it is receiving a persistent excitatory input sequence. After each refractory period, the neuron moves to location $\mathbf{A}_H$, instead of $\mathbf{A}$. The only difference between $\mathbf{A}_H$ and $\mathbf{A}$ is that $\mathbf{A}_H$ ignores positive values of a at the end

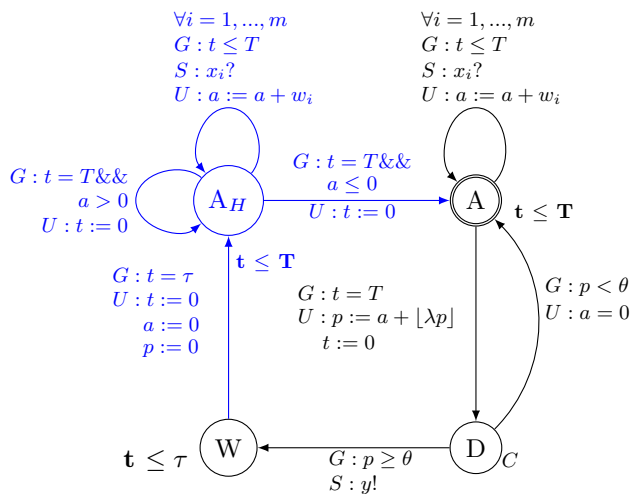


Fig. 4 The extended neuron model for phasic spiking. Additions are colored in blue. (Color figure online)

of each accumulation period. Conversely, a non-positive value of a (denoting the end of the persistent input), at the end of some accumulation period, leads the neuron back in location A .

4.5 Bursting

A *burst* is a finite sequence of *high frequency* spikes. More formally:

Definition 8 A spike output sequence is a *burst* if it is composed by spikes having an occurrence rate greater than $1/\tau$, with τ being the refractory period of the neuron.

A *burst sequence* is a sequence composed by bursts, subject to the following constraint: the time difference between the last spike of each burst and the first spike of the next burst is greater than τ .

Property 8 *Neurons cannot produce bursts.*

Proof A neuron $\mathcal{N}$ cannot emit spikes having a rate greater than $1/(T + \tau)$, as stated by Property 2, so it cannot produce bursts. $\square$

In order to reproduce bursts our model can be extended by allowing several subsequent emissions in an interval period smaller than τ . After this period all clocks and variables are reset and the accumulation-fire-rest cycle can start again.

Several bursting behaviours are described in Izhikevich (2004). Here we discuss only three of them, as all impossibility results depend on Property 8 and all the automata extensions are similar.

Tonic Bursting is the behaviour of a neuron producing a burst sequence as a response to a persistent and excitatory input sequence. *Phasic Bursting* is the behaviour of a neuron

producing a burst as a consequence of a persistent excitatory input sequence and then remaining quiescent. Obviously the preceding behaviours require the ability of producing bursts.

Bursting-then-Spiking is the behaviour of a neuron producing a burst as response to a persistent excitatory input sequence and then producing a periodic output sequence. Such a behaviour, similarly to Phasic and Tonic Bursting, depends on the neuron ability of producing bursts. Moreover it requires inter-emission memory, in order to detect the beginning of a persistent sequence.

Property 9 *Neurons cannot exhibit the Tonic Bursting, Phasic Bursting and Bursting-then-Spiking behaviours.*

Proof Follows from Property 8. $\square$

4.6 Spike frequency adaptation

Spike Frequency Adaptation is the behaviour of a neuron producing a decreasing-frequency output sequence as a response to a persistent excitatory input sequence. In other words, the inter-emission time difference increases as the time elapses. This behaviour requires the neuron to have inter-emission memory as it should be able to keep track of the time elapsed since the beginning of the input sequence.

Property 10 *Neurons cannot reproduce the Spike Frequency Adaptation behaviour.*

Proof The Spike Frequency Adaptation behaviour requires the neuron to detect the beginning of an excitatory input sequence and to increase the time required to fire a spike, after each emission. This means the neuron will produce different outcomes as response to equal inputs, which is impossible, as stated in Property 3. $\square$

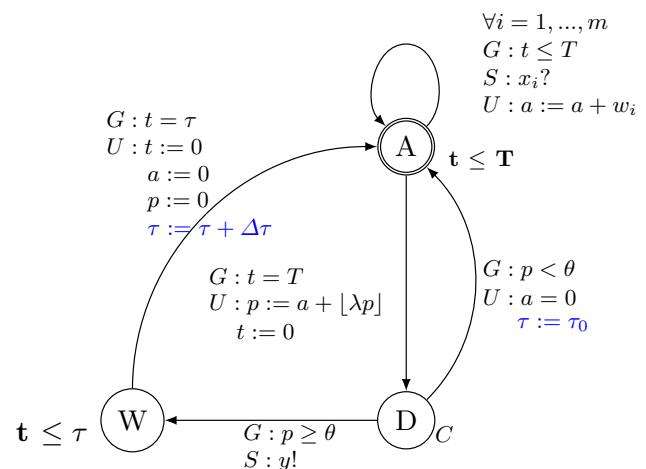


Fig. 5 The extended model for spike frequency adaptation behaviour. Additions are colored in blue. (Color figure online)

An extended neuron model able to reproduce Spike Frequency Adaptation behaviour is shown in Fig. 5. This variant allows the refractory period to increase after each neuron emission, thus making the output frequency decrease.

4.7 Spike latency

Spike Latency is the behaviour of a neuron firing delayed spikes, with respect to the instant when its potential reached or overcame the threshold. Such a delay is proportional to the strength of the signal which leads it to emission, i.e., the sum of weighed inputs received during the accumulation period preceding the emission. This behaviour requires the neuron to be able to postpone its output.

Property 11 *Neurons cannot reproduce the Spike Latency behaviour.*

Proof The property holds by construction. As location **D** is *committed*, no firing can be delayed. $\square$

An easy solution to extend our model is to introduce a delay between the instant the neuron reaches or overcomes its threshold and the actual emission instant. Such a delay δ depends on the sum of weighted inputs received during the last accumulation period. If the potential is greater than or equal to the threshold, the neuron computes the delay duration $\delta(a)$, assigning it to an integer variable d , and then waits in location **Del** for d time units before emitting a spike on channel y . The extended version is depicted in Fig. 6.

4.8 Threshold variability

Threshold variability is the behaviour of a neuron allowing its threshold to vary according to the strength of its inputs.

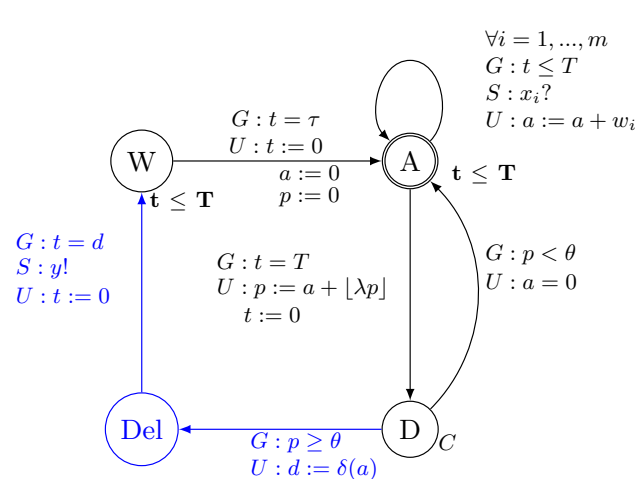


Fig. 6 The extended model for the spike latency behaviour. Additions are colored in blue. (Color figure online)

More precisely, an excitatory input will rise the threshold while an inhibitory input will decrease it. As a consequence, excitatory inputs may more easily lead the neuron to fire when occurring after an inhibitory input.

Property 12 *Neurons cannot reproduce the Threshold Variability behaviour.*

Proof By construction the neuron threshold never changes. $\square$

The neuron model can be extended allowing the threshold to vary after each accumulation period according to the current sum of weighted inputs (see Fig. 7). The threshold variable initial value is θ_0 . On every firing of edge (**A** $\rightarrow$ **D**), the threshold variable is increased of $\Delta(a)$, where a is the sum of weighted inputs occurred during the last accumulation period and $\Delta(a)$ is an integer value whose sign is opposite to the sign of a and whose magnitude is proportional to the magnitude of a .

4.9 Bistability

Bistability is the behaviour of a neuron alternating between two operation modes: *periodic emission* and *quiescence*. Upon reception of a single excitatory spike, it emits a periodic output sequence and switches to a quiescent mode (no emission) as soon as it received another spike. Such a behaviour requires the neuron to (1) be able to produce a periodic output sequence, even if no excitatory spike is received, (2) be able to remain silent when no spike is received, and (3) be able to switch between the two operation modes upon reception of an excitatory spike.

Property 13 *Neurons cannot reproduce the Bistability behaviour.*

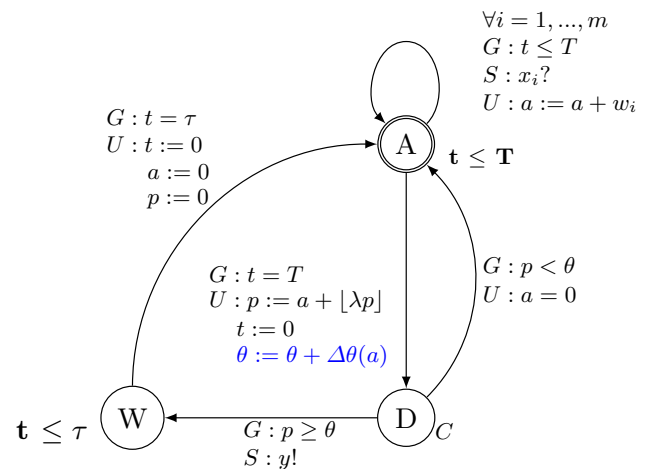


Fig. 7 The extended model for the threshold variability behaviour. Additions colored are in blue. (Color figure online)

Proof The only possibility of obtaining a periodic output as a result of no excitatory input spike is to set $\theta = 0$. This is a limit case of Property 4. Since, by construction, the threshold cannot vary, the neuron cannot switch between the two operation modes. $\square$

The neuron model can be modified as shown in Fig. 8. This variant makes its threshold switch between 0 and a positive value at the end of any accumulation period during which it received an excitatory sum of weighted inputs a . A null threshold would make the neuron emit even if no input is received. Conversely, a positive threshold would prevent the neuron from emitting, if no input is received. Thus, on every firing of edge $(A \rightarrow D)$, the threshold value θ is computed by the function $bist(\cdot)$:

$$bist(\theta, a) = \begin{cases} 0 & \text{if } \theta > 0 \wedge a > 0 \\ \theta_0 & \text{if } \theta = 0 \wedge a > 0 \\ \theta & \text{if } a \leq 0. \end{cases}$$

4.10 Inhibition-induced activities

This is the behaviour of a neuron producing a spike output sequence as a response to a persistent *inhibitory* input sequence. We thus require the neuron to be able to emit as a consequence of some inhibitory input spikes.

Property 14 *Neurons cannot reproduce the Inhibition-induced Spiking behavior.*

Proof Follows from Fact 1. $\square$

An easy extension to our automata is to consider the *absolute value* of all inputs instead of their signed values.

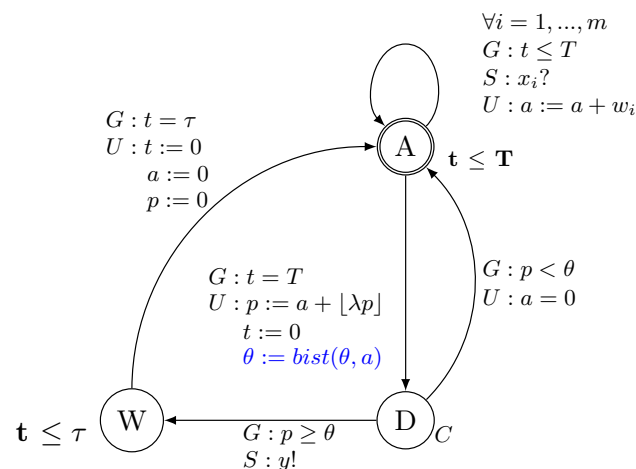


Fig. 8 The extended model for bistability behaviour additions are colored in blue. (Color figure online)

4.11 Rebound activities

Rebound Spike is the behaviour of a neuron producing an output spike after it received an inhibitory input. Similarly to Inhibition-induced activities, this behaviour requires the neuron to emit as a consequence of an inhibitory input spike.

Property 15 *Neurons cannot exhibit the Rebound Spiking behaviour.*

Proof Follows from Fact 1. $\square$

We can modify our encoding by setting the neuron potential to be always non-negative and by fixing the threshold to be 0 as response to an inhibitory stimulation. Recall that a null threshold would make the neuron emit even if its potential is 0. Thus, on every firing of the edge $(A \rightarrow D)$, if the current sum of weighted inputs a is negative, the threshold θ is set to 0, otherwise it is set to a $\theta > 0$. This will allow an inhibitory stimulus to produce a rebound spike.

5 Parameter inference

In this section we examine the *Learning Problem*: i.e., how to determine a parameter assignment for a network with a fixed topology and a given input such that a desired output behaviour is displayed. Here we only focus on the estimation of synaptic weights in a given spiking neural network; the generalisation of our methodology to other parameters is left for future work.

Our analysis takes inspiration from the SpikeProp algorithm (Bohte et al. 2002), which deals with fully connected feedforward networks of spiking neurons with layers and aims at attaining a set of target firing times of the output neurons for a given set of input patterns. An error function is obtained by computing the least mean squares of desired spike times and actual firing times; such an error is then back-propagated in the network. In a similar way, we detect the errors of output neurons and back-propagate them. The main differences between the SpikeProp approach and our ones are: (1) the SpikeProp rule deals with multi-layered cycle-free continuous spiking neural networks while our networks are discrete and not multi-layered; (2) the Spikeprop rule does not treat negative edges (for this, inhibitory and excitatory neurons are introduced) while our connections are allowed to contain a mix of both positive and negative edges.

In the technique we propose here, the learning process is led by *supervisors*. Differently from the previous section, each output neuron $\mathcal{N}$ is linked to a supervisor instead of an output consumer. Supervisors compare the expected

output behaviour with the one of the output neuron they are connected to (function $\text{EVALUATE}(\mathcal{N})$ in Algorithm 1). Thus either the output neuron behaved consistently or not. In the second case and in order to instruct the network, the supervisor back-propagates *advices* to the output neuron depending on two possible scenarios: (1) the neuron fires a spike, but it was supposed to be quiescent, (2) the neuron remains quiescent, but it was supposed to fire a spike. In the first case the supervisor addresses a *should not have fired* message (SNHF) and in the second one a *should have fired* (SHF). Then each output neuron modifies its ingoing synaptic weights and in turn behaves as a supervisor with respect to its predecessors, back-propagating the proper advice.

The advice back-propagation (ABP), Algorithm 1, basically lies on a depth-first visit of the graph topology of the network. Let $\mathcal{N}_i$ be the i -th predecessor of an automaton $\mathcal{N}$, then we say that $\mathcal{N}_i$ fired, if it emitted a spike during the current or previous accumulate-fire-wait cycle of $\mathcal{N}$. Thus, upon reception of a SHF message, $\mathcal{N}$ has to *strengthen* the weight of each ingoing *excitatory* synapse and *weaken* the weight of each ingoing *inhibitory* synapse. Then, it propagates a SHF advice to each ingoing *excitatory* synapse (i.e., an arc with weight greater than 0: $\text{WT} \geq 0$) corresponding to a neuron which *did not* fire recently ($\neg F(\mathcal{N})$), and symmetrically a SNHF advice to each ingoing *inhibitory* synapse ($\text{WT} < 0$) corresponding to a neuron which fired recently (see Algorithm 2 for SHF, and Algorithm 3 for the dual case of SNHF). When the graph visit reaches an input generator, it will simply ignore any received advice (because input sequences should not be affected by the learning process). The learning process ends when all supervisors do not detect any more errors.

Algorithm 1 The advice back-propagation algorithm

```

1: function ABP
2:   discovered =  $\emptyset$ 
3:   for all  $\mathcal{N} \in \text{Output}$  do
4:     if  $\mathcal{N} \notin \text{discovered}$  then
5:       discovered = discovered  $\cup \mathcal{N}$ 
6:       if  $\text{EVALUATE}(\mathcal{N}) = \text{SHF}$  then
7:         SHF( $\mathcal{N}$ )
8:       else if  $\text{EVALUATE}(\mathcal{N}) = \text{SNHF}$  then
9:         SNHF( $\mathcal{N}$ )

```

Algorithm 2 Should Have Fired algorithm

```

1: procedure SHOULD-HAVE-FIRED( $\mathcal{N}$ )
2:   if  $\mathcal{N} \in \text{discovered} \cup \text{Output}$  then
3:     return
4:   discovered = discovered  $\cup \mathcal{N}$ 
5:   for all  $\mathcal{M} \in \text{PRED}(\mathcal{N})$  do
6:     if  $\mathcal{M} \notin \text{Input}$  then
7:       if  $\text{WT}(\mathcal{M}, \mathcal{N}) \geq 0 \wedge \neg F(\mathcal{M})$  then
8:         SHF( $\mathcal{M}$ )
9:       if  $\text{WT}(\mathcal{M}, \mathcal{N}) < 0 \wedge F(\mathcal{M})$  then
10:        SNHF( $\mathcal{M}$ )
11:   INCREASE-WEIGHT( $\mathcal{M}, \mathcal{N}$ )
12: return

```

Algorithm 3 Abstract ABP: Should Not Have Fired advice pseudo-code

```

1: procedure SHOULD-NOT-HAVE-FIRED(neuron)
2:   if  $\mathcal{N} \in \text{discovered} \cup \text{Output}$  then
3:     return
4:   discovered = discovered  $\cup \mathcal{N}$ 
5:   for all  $\mathcal{M} \in \text{PREDECESSORS}(\mathcal{N})$  do
6:     if  $\mathcal{M} \notin \text{Input}$  then
7:       if  $\text{WT}(\mathcal{M}, \mathcal{N}) \geq 0 \wedge F(\mathcal{M})$  then
8:         SNHF( $\mathcal{M}$ )
9:       if  $\text{WT}(\mathcal{M}, \mathcal{N}) < 0 \wedge \neg F(\mathcal{M})$  then
10:        SHF( $\mathcal{M}$ )
11:   DECREASE-WEIGHT( $\mathcal{M}, \mathcal{N}$ )
12: return

```

There are several possibilities on how to realise supervisors and the ABP algorithm. We propose here two approaches. The first one is model checking oriented and it is based on the idea that supervisors are represented by temporal logic formulae. The second one is simulation oriented, and the implementation of the algorithm is embedded into the timed automata modelling of the neuron.

5.1 Model-checking-oriented approach

Such a technique consists in iterating the learning process until a desired CTL property concerning the output of the network is verified. The hypothesis we introduce are the following ones: (1) input generators, standard neurons, and output consumers share a global clock which is never reset and (2) for each output consumer, there exists a clock measuring the elapsed time since the last received spike. The CTL formula specifying the expected output of the network can only contain predicates relative to the output consumers and the global clock. At each step of the algorithm, we make an external call to the model checker to test whether the network satisfies the formula or not. If the formula is verified, the learning process ends; otherwise, the model checker provides a trace as a counterexample. Such a trace is exploited to derive the proper corrective action to be applied to each output neuron, that is, the invocation of either the SHF procedure, or the SNHF procedure previously described (or no procedure).

More in detail, given a timed automata network representing some spiking neural network, we extend it with a global clock t_g which is never reset and, for each output consumer $\mathcal{O}_K$ relative to the output neuron $\mathcal{N}_k$, we add a clock s_k measuring the time elapsed since the last spike consumed by $\mathcal{O}_k$. Furthermore, let $\text{state}_{\mathcal{O}_k}(\mathbf{O})$ be an atomic proposition evaluating to true if the output consumer $\mathcal{O}_K$ is in its $\mathbf{O}$ location, and let $\text{eval}_{\mathcal{O}_k}(s_k)$ be an atomic proposition indicating the value of the clock s_k in $\mathcal{O}_K$. In order to make it possible to deduce the proper corrective action, we

impose the CTL formula describing the expected outcome of the network to be composed by the conjunction of sub-formulae respecting any of the patterns presented in the following.

Precise firing The output neuron $\mathcal{N}_k$ fires at time t :

$$AF(t_g = t \wedge state_{\mathcal{O}_k}(\mathbf{O})).$$

The violation of such a formula requires the invocation of the SHF procedure.

Weak quiescence The output neuron $\mathcal{N}_k$ is quiescent at time t :

$$AG(t_g = t \Rightarrow \neg state_{\mathcal{O}_k}(\mathbf{O})).$$

The SNHF procedure is called in case this formula is not satisfied.

Relaxed firing The output neuron $\mathcal{N}_k$ fires at least once within the time window $[t_1, t_2]$:

$$AF(t_1 \leq t_g \leq t_2 \wedge state_{\mathcal{O}_k}(\mathbf{O})).$$

The violation of such a formula leads to the invocation of the SHF procedure.

Strong quiescence The output neuron $\mathcal{N}_k$ is quiescent for the whole duration of the time window $[t_1, t_2]$:

$$AG(t_1 \leq t_g \leq t_2 \Rightarrow \neg state_{\mathcal{O}_k}(\mathbf{O})).$$

The SNHF procedure is needed in this case.

Precise periodicity The output neuron $\mathcal{N}_k$ eventually starts to periodically fire a spike with exact period P :

$$AF(AG(eval_{\mathcal{O}_k}(s_k) \neq P \Rightarrow \neg state_{\mathcal{O}_k}(\mathbf{O})) \wedge AG(state_{\mathcal{O}_k}(\mathbf{O}) \Rightarrow eval_{\mathcal{O}_k}(s_k) = P)).$$

If $\mathcal{N}_k$ fires a spike while the s_k clock is different than P or it does not fire a spike while the s_k clock equals P , the formula is not satisfied. In the former (resp. latter) case, we deduce that the SNHF (resp. SHF) procedure is required.

Relaxed periodicity The output neuron $\mathcal{N}_k$ eventually begins to periodically fire a spike with a period that may vary in $[P_{min}, P_{max}]$:

$$AF(AG(eval_{\mathcal{O}_k}(s_k) \notin [P_{min}, P_{max}]) \Rightarrow$$

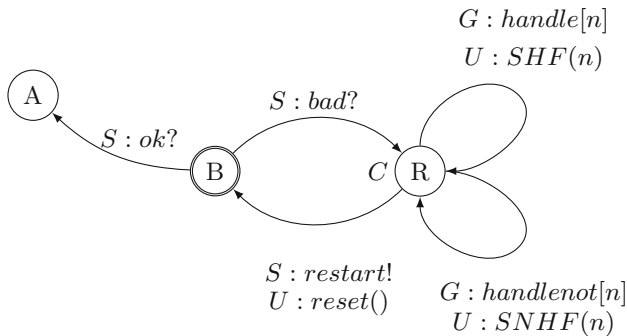


Fig. 9 The automata responsible of the ABP-alg

$$\neg state_{\mathcal{O}_k}(\mathbf{O})) \wedge AF(state_{\mathcal{O}_k}(\mathbf{O}) \Rightarrow P_{min} \leq eval_{\mathcal{O}_k}(s_k) \leq P_{max})).$$

For the corrective actions, see the previous case.

As for future work, we intend to extend this set of CTL formulae with new formulae concerning the comparison of the output of two or more given neurons. Please notice that the Uppaal model-checker only supports a fragment of CTL where the use of nested path quantifiers is not allowed. Another model-checker should be called in order to fully exploit the expressive power of CTL.

5.2 Simulation-oriented approach

In this second approach, parameters are modified during the simulation of the network. This entails that the encoding of neurons needs to be adjusted in order to take care of the adaptation of such parameters. Algorithm ABP is realised by a dedicated automaton $ABP - alg$, that is depicted in Fig. 9, and the role of supervisor is given to output consumers, that are modified as in Fig. 10.

The idea is that, according to the function EVALUATE, if the corresponding output neuron misbehaves, then its output consumer sets whether it has to be treated according to the SHF or the SNHF function. Furthermore, it signals to the $ABP - alg$ through the message *bad!* that some adjustments on the network have to be done. Then the $ABP - alg$ automaton takes the lead and it recursively applies the function SHF or SNHF (this is achieved by setting a proper variable in a vector named *handle*) on the predecessors of the output neuron. Once there is no more neuron to whom the algorithm should be applied (for instance all neurons in the current run have been visited), the simulation is restarted in the network with the new parameters. If the output consumer does not recognise any misbehaviour, than it sends an *ok!* message to the $ABP - alg$ automaton, that in turn moves to an accepting state A.

More formally:

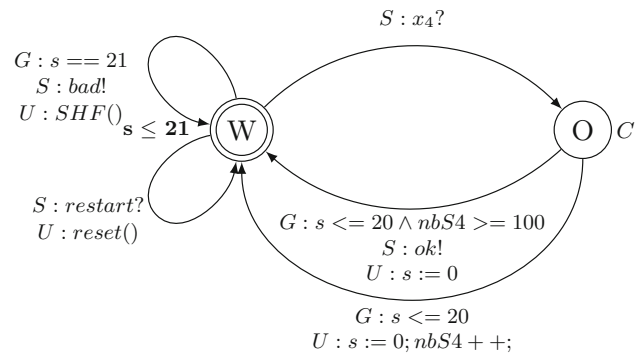


Fig. 10 Example of an output consumer in the simulation approach for the diamond structure

Definition 9 (*Output consumer for the simulation approach*) An output consumer for the simulation approach is a timed automaton

$$\mathcal{N} = (L, \mathbf{A}, X, \text{Var}, \Sigma, \text{Arcs}, \text{Inv})$$

with

- $L = \{\mathbf{W}, \mathbf{O}\}$ with $\mathbf{O}$ committed,
- $X = \{s\}$
- $\text{Var} = \{nb, \text{handle}[N], \text{handlenot}[N]\}$
- $\Sigma = \{x_i \mid x_i \text{ is an output neuron}\} \cup \{\text{bad}, \text{restart}, \text{ok}\},$
- $\text{Arcs} = \{(\mathbf{W}, s < T_{\min}, \text{bad!}, \{\text{SHF}()\}, \mathbf{W}), (\mathbf{W}, \text{restart?}, \{s := 0, nb := 0\}, \mathbf{W}), (\mathbf{W}, x_i?, \mathbf{O}), (\mathbf{O}, s > T_{\max}, \text{bad!}, \{\text{SNHF}()\}, \mathbf{W}), (\mathbf{O}, \text{restart?}, \{s := 0, nb := 0\}, \mathbf{O}), (\mathbf{O}, \text{good_pattern}, \text{ok!}, \{s := 0\}, \mathbf{W}), (\mathbf{O}, \text{good_not_finished}, \{s := 0\}, \mathbf{W})\};$
- $\text{Inv}(\mathbf{W}) = s \leq T, \text{Inv}(\mathbf{O}) = \text{true}$

where the functions SHF and SNHF modify the global variables handle_i and handlenot_i respectively, that are used in the *ABP - alg* automaton.

Notice that the precise definition (for instance the value of parameters good_pattern , good_not_finished , $T_{\min}$, $T_{\max}$) of the output consumer depends on the expected behaviour of the supervisor. As an example, in Fig. 10, that depicts the output consumer for the diamond network (see the details in the next subsection), we expect the output neuron to emit a spike within each window of 20 seconds for at least 100 times.

More in detail, the cycle from $\mathbf{W}$ to $\mathbf{W}$ is taken whenever a spike has not been sent before T time units. Thus the automaton sends to the *ABP - alg* automaton a *bad* message (signalling that an adjustment to the network should take place), and the update part handles the fact that we should perform the SHF algorithm (a variable corresponding to the concerned output neuron is set to **true** in

the array *handle*). From the same state, whenever the message *restart* is received, all the variables of the automaton are reset to 0. When a spike from the output neuron x is received, the output consumer moves to the state $\mathbf{O}$. In this state, if the spike was received too late (and similarly as in the previous case), a *bad* message is sent to the *ABP - alg*. Otherwise, if everything was received on time and if the expected pattern has been completely verified, then an *ok* message is sent and the automaton moves to the initial state (Fig. 11).

In the following, we give the formal definition of the *ABP - alg* automaton:

Definition 10 (*ABP - alg automaton*)

- $L = \{\mathbf{A}, \mathbf{B}, \mathbf{R}\}$ with $\mathbf{R}$ committed,
- $X = \emptyset$
- $\text{Var} = \{\text{handle}[N], \text{handlenot}[N]\}$
- $\Sigma = \{\text{bad}, \text{ok}, \text{restart}\},$
- $\text{Arcs} = \{(\mathbf{B}, \text{ok?}, \mathbf{A}), (\mathbf{B}, \text{bad?}, \mathbf{R}), (\mathbf{R}, \text{handle}[n], \text{SHF}(n), \mathbf{R}), (\mathbf{R}, \text{handlenot}[n], \text{SNHF}(n), \mathbf{R}), (\mathbf{R}, \text{finished?}(), \text{restart!}, \text{reset}(), \mathbf{B})\};$
- $\text{Inv}(\mathbf{A}) = \text{true}, \text{Inv}(\mathbf{B}) = \text{true}, \text{Inv}(\mathbf{R}) = \text{true}.$

The arc from the state $\mathbf{B}$ to the accepting state $\mathbf{A}$ is taken whenever the Output consumer has finished the analysis of its pattern. Conversely, the arc from the state $\mathbf{B}$ to $\mathbf{R}$ is adopted when one of the output neurons has misbehaved (signalled by the reception of the message *bad*). In the committed state $\mathbf{R}$, the parameters of the neural network are changed accordingly to the Algorithms 2 and 2. The arrays *handle* (respectively *handlenot*) have the information on the neurons to which the Algorithm for SHF (respectively SNHF) has to be applied. Once the cycle of updates finishes (checked through the function *finished*), the simulation is restarted by broadcasting a message *restart* to all the neurons and the output consumer.

Last, we give the definition of the changes induced in the standard neuron. As the update of the neuron parameters is done at the level of *ABP - alg*, the only change concerns the treatment of the signal *restart*. To this aim an arc handling the reception of the message is added to the states $\mathbf{W}$ and $\mathbf{A}$.

Definition 11 (*Standard neuron for the simulation approach*) Given a neuron $v = (\theta, \tau, \lambda, p, y)$ with m input synapses, its encoding into timed automata is $\mathcal{N} = (L, \mathbf{A}, X, \text{Var}, \Sigma, \text{Arcs}, \text{Inv})$ with:

- $L = \{\mathbf{A}, \mathbf{W}, \mathbf{D}\}$ with $\mathbf{D}$ committed,
- $X = \{t\}$
- $\text{Var} = \{p, a\}$
- $\Sigma = \{x_i \mid i \in [1..m]\} \cup \{y\},$

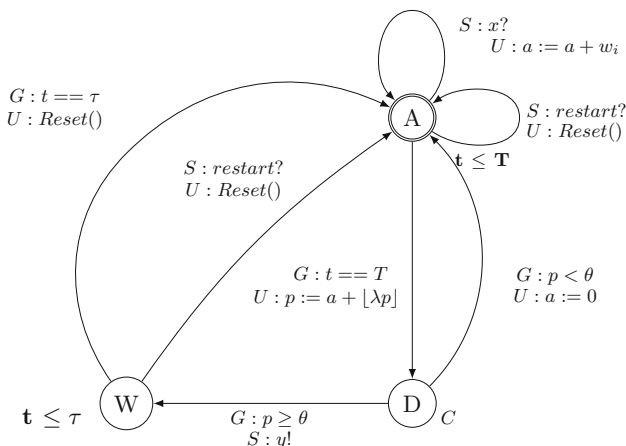


Fig. 11 Example of a neuron in the simulation approach for the diamond network

- $Arcs = \{(\mathbf{A}, t \leq T, x_i?, \{a := a + w_i\}, \mathbf{A}) \mid i \in [1..m]\} \cup \{(\mathbf{A}, t = T, \{p := a + \lfloor \lambda p \rfloor\}, \mathbf{D}), (\mathbf{D}, p < \theta, \{a := 0\}, \mathbf{A}), (\mathbf{D}, p \geq \theta, y!, \mathbf{W}), (\mathbf{W}, t = \tau, \{a := 0, t := 0, p := 0\}, \mathbf{A}), (\mathbf{A}, restart?, \{a'' = 0, t := 0, p := 0\}, \mathbf{A}), (\mathbf{W}, restart?, \{a'' = 0, t := 0, p := 0\}, \mathbf{A})\}$;
- $Inv(\mathbf{A}) = t \leq T, Inv(\mathbf{W}) = t \leq \tau, Inv(\mathbf{D}) = \text{true}$.

Notice that the algorithm can be refined by setting some priorities on the neurons. For instance, if any additional information is known in advance on the behaviour of a specific neuron, its actions can be constrained and the corrective operations could be ignored. In some cases, the type of synapse (excitatory or inhibitory) can also be set, disallowing the possibility of changing its nature (e.g., from inhibitory to excitatory).

Notice that, as the application of the ABP algorithm in the simulation approach is non-deterministic, the parameters we found may depend on the precise execution and several solutions are therefore possible.

5.3 Examples

In this section we show several examples where we have applied our approaches to determine a parameter assignment. The complete encoding of the examples shown here can be found at De Maria et al. (2017b).

5.4 Turning on and off a diamond structure of neurons

This example shows how the ABP algorithm can be used to make a neuron emit at least once in a spiking neural

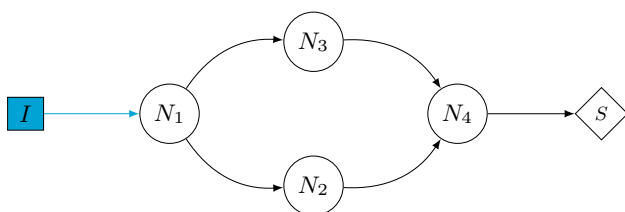


Fig. 12 A neural network with a diamond structure

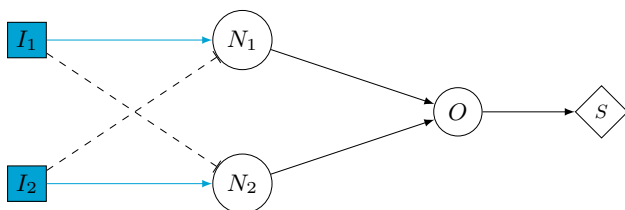


Fig. 13 Structure of the XOR network. Dashed (resp. continuous) edges stand for inhibitions (resp. activations)

network having the *diamond* structure shown in Fig. 12. We assume that $\mathcal{N}_1$ is fed by an input generator $\mathcal{I}$ that continuously emits spikes. No neuron in the network is able to emit because all the weights of their input synapses are equal to zero and their thresholds are higher than zero. The initial weights and parameters are:

$w_{0,1}$	$w_{1,2}$	$w_{1,3}$	$w_{2,4}$	$w_{3,4}$
0.1	0.1	0.1	0.1	0.1

Neuron	T	θ	τ	λ
$\mathcal{N}_1$	2	0.35	3	7/9
$\mathcal{N}_2$	2	0.35	3	7/9
$\mathcal{N}_3$	2	0.35	3	7/9
$\mathcal{N}_4$	2	0.55	3	1/2

We want the network to learn a weight assignment so that $\mathcal{N}_4$ is able to emit, that is, to produce a spike after an initial pause.

At the beginning we expect no activity from neuron $\mathcal{N}_4$. As soon as the initial pause is elapsed, we require a spike but, as all weights are equal to zero, no emission can happen. Thus a SHF advice is back-propagated to neurons $\mathcal{N}_2$ and $\mathcal{N}_3$ and consequently to $\mathcal{N}_1$. The process is then repeated until all weights stabilise and neuron $\mathcal{N}_4$ is able to fire.

Here we apply the *model-checking approach*. We expect $\mathcal{N}_4$ to spike every 20 time units. After three cycles of the algorithm (i.e., three checks of the formula and modifications of weights), we reach the following weight assignment:

$w_{0,1}$	$w_{1,2}$	$w_{1,3}$	$w_{2,4}$	$w_{3,4}$
0.3	0.3	0.3	0.3	0.3

5.5 The XOR problem

The XOR problem is a classic problem in neural networks. It is used to classify inputs interpreting the outputs as the result of the logic XOR between the inputs (a logic XOR returns true if the two inputs are not equal and false

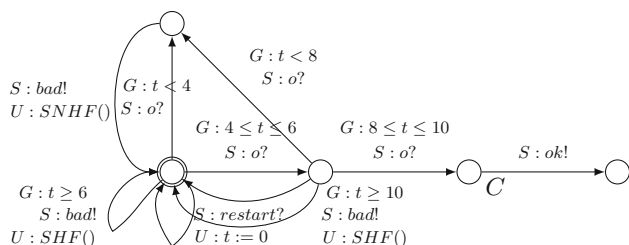


Fig. 14 The output consumer for the XOR network

otherwise). The problem is interesting because it can be solved only within a multilayered architecture. Figure 13 shows an example of such structure. Although typical in neural networks, this example is not adapted to our setting. Our networks expect as input streams of spikes and, as mentioned in the introduction, we do not aim at classifying specific inputs.

To adjust the XOR problem to our setting, we consider a network where neurons have no memory (the leak factor is 0) and the refractory period is also 0. As input we consider two finite sequences $I_1 = P(8) \cdot s \cdot P(3) \cdot s$ and $I_2 = P(4) \cdot s \cdot P(7) \cdot s$ encoding the sequence of bits 0 0 1 1 for I_1 and 0 1 0 1 for I_2 . The expected output is $S = P(5) \cdot s \cdot P(4) \cdot s$ encoding the sequence of bits 0 1 1 0. Figure 14 shows the realisation of the output consumer. The other parameters are $T = 1$ and $\Theta = 1$ while the weights are

$w_{I_1,1}$	$w_{I_1,2}$	$w_{I_2,1}$	$w_{I_2,2}$	$w_{1,O}$	$w_{2,O}$
0.1	- 0.1	0.1	- 0.1	0.1	0.1

With the *simulation approach* we reach the following assignment:

$w_{I_1,1}$	$w_{I_1,2}$	$w_{I_2,1}$	$w_{I_2,2}$	$w_{1,O}$	$w_{2,O}$
1	- 0.1	1	- 0.1	1	1

5.6 A negative loop

In this example, we show a network that is meant to oscillate and we use our learning algorithm to find parameters for a specific oscillation period. The network is



Fig. 15 A simple negative loop network

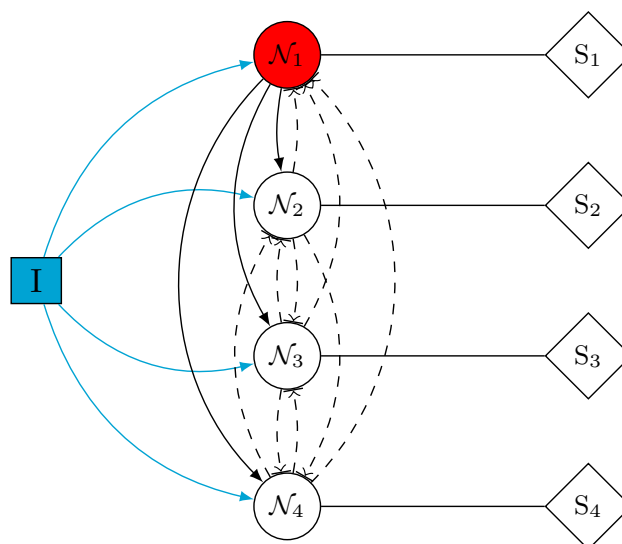


Fig. 16 We denote neurons by $\mathcal{N}_i$. The network is fed by an input generator $\mathcal{I}$ and the learning process is led by the supervisors S_i

showed in Fig. 15, it consists of two neurons N_1 and N_2 : the first one excites the second one while the second one inhibits the first one. We expect N_2 to fire each 3 time units.

In this example we use a *mixture of the two approaches* described above. We model the network following the simulation approach and we check the formula $AG \neg \text{state}_{ABP}(\mathbf{A})$ stating that it is not possible to reach the accepting state in the ABP automaton. The formula, as expected, is false and the counterexample provides a weight assignment.

For both neurons, we have $T = 1$, $\theta = 1$, $\lambda = 1$, and no refractory period. The initial weights are:

$w_{I,1}$	$w_{1,2}$	$w_{2,1}$
0.3	0.5	- 0.7

and the counterexample provides the following weights:

$w_{I,1}$	$w_{1,2}$	$w_{2,1}$
0.5	1	- 0.5

5.7 Mutual inhibition networks

The last example is about mutual inhibition networks, where the constituent neurons inhibit each other neuron's

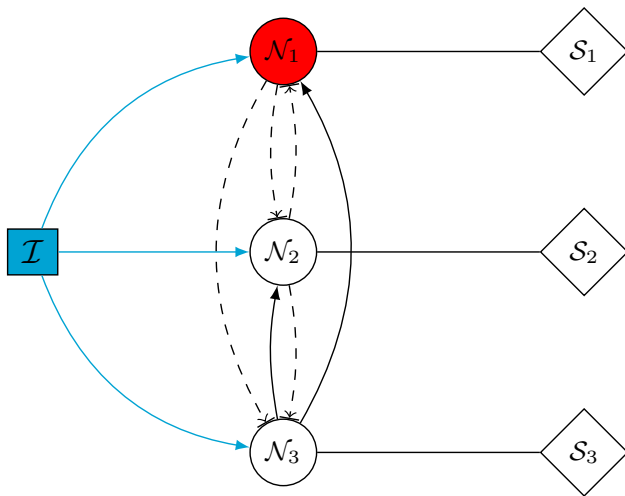


Fig. 17 A second neural network with “winner takes all” behaviour

activity. These networks belong to the set of Control Path Generators (CPGs), which are known for their capability to produce rhythmic patterns of neural activity without receiving rhythmic inputs (Ijspeert 2008). CPGs underlie many fundamental rhythmic activities such as digesting, breathing, and chewing. They are also crucial building blocks for the locomotor neural circuits both in invertebrate and vertebrate animals. It has been observed that, for suitable parameter values, mutual inhibition networks present a behaviour of the kind “winner takes all”, that is, at a certain time one neuron becomes (and stays) activated and the other ones are inhibited (De Maria et al. 2016). These networks are similar to the continuous switch networks used in modelling of the development (Cinquin and Demongeot 2005).

We consider a mutual inhibition network of four neurons, as shown in Fig. 16. This example, although being small, it is not trivial as it features inhibitor and excitatory edges as well as cycles.

We look for synaptical weights such that the “winner takes all” behaviour is displayed. We assume each neuron to be fed by an input generator $\mathcal{I}$ that continuously emits spikes. At the beginning, all the neurons have the same parameters (that is, firing threshold, remaining coefficient, accumulation period, and refractory period), and the weight of excitatory (resp. inhibitory) edges is set to 1 (resp. -1). We use the ABP algorithm with the *model checking approach* to learn a weight assignment so that the first neuron is the winner. More precisely, we find a weight assignment such that, whatever the chosen path in the corresponding automata network is, the network stabilises when the global clock t_g equals 70. The weight of the edges from the input generator $\mathcal{I}$ to the four neurons equals 0.041. The weight of the edges inhibiting $\mathcal{N}_1$ (resp. $\mathcal{N}_2$, $\mathcal{N}_3$, and $\mathcal{N}_4$) is -0.719 (resp. -0.817).

For the *simulation approach*, we used a smaller network with 3 neurons only, depicted in Fig. 17. As before, we require a “winner takes all” behaviour where $\mathcal{N}_1$ is the winner. We expect that $\mathcal{N}_1$ spikes every 10 time units at most, and the neurons $\mathcal{N}_2$ and $\mathcal{N}_3$ do not spike. We consider a network with the following initial parameters:

Neuron	T	θ	τ	λ
$\mathcal{N}_1$	2	0.75	3	1/2
$\mathcal{N}_2$	2	0.75	3	1/2
$\mathcal{N}_3$	2	0.75	3	1/2

$w_{x,y}$	$\mathcal{I}$	$\mathcal{N}_1$	$\mathcal{N}_2$	$\mathcal{N}_3$
$\mathcal{I}$	0	0.1	0.1	0.1
$\mathcal{N}_1$	0	0	-0.1	-0.1
$\mathcal{N}_2$	0	-0.1	0	-0.1
$\mathcal{N}_3$	0	0.1	0.1	0

We obtain the following weights in 5 executions of the ABP algorithm:

$w_{x,y}$	$\mathcal{I}$	$\mathcal{N}_1$	$\mathcal{N}_2$	$\mathcal{N}_3$
$\mathcal{I}$	0	0.6	0.1	0.1
$\mathcal{N}_1$	0	0	-0.1	-0.1
$\mathcal{N}_2$	0	-0.1	0	-0.1
$\mathcal{N}_3$	0	0.6	0.1	0

6 Related work

To the best of our knowledge, there are few attempts of giving formal models for LI&F. Apart from the already discussed approach of De Maria et al. (2016), where the authors model and verify LI&F networks thanks to the synchronous language Lustre, the closest related work we are aware of is Aman and Ciobanu (2016). In this work, the authors propose a mapping of spiking neural P systems into timed automata. The modelling is substantially different from ours. They consider neurons as static objects and the dynamics is given in terms of evolution rules while for us the dynamics is intrinsic to the modelling of the neuron. This, for instance, entails that inhibitions are not just negative weights as in our case, but are represented as *forgetting rules*. On top of this, the notion of time is also

different: while they consider durations in terms of number of applied rules, we have an explicit notion of duration given in terms of accumulation and refractory period.

As far as our parameter learning approach is concerned, we borrow inspiration from the SpikeProp rule (Bohte et al. 2002), a variant of the well known back-propagation algorithm (Rumelhart et al. 1988) used for supervised learning in second generation learning. The SpikeProp rule deals with multi-layered cycle-free spiking neural networks and aims at training networks to produce a given output sequence for each class of input sequences. The main difference with respect to our approach is that we are considering here a discrete model and our networks are not multi-layered. We also rest on Hebb's learning rule (Hebb 1949) and its time-dependent generalisation rule, the spike timing dependent plasticity (STDP) rule (Sjöström 2010), which aims at adjusting the synaptical weights of a network according to the time occurrences of input and output spikes of neurons. It acts locally, with respect to each neuron, i.e., no prior assumption on the network topology is required in order to compute the weight variations for some neuron input synapses. Differently from the STDP, our approach takes into account not only recent spikes but also some external feedback (*advices*) in order to determine which weights should be modified and whether they must increase or decrease. Moreover, we do not prevent excitatory synapses from becoming inhibitory (or vice versa), which is usually a constraint for STDP implementations. A general overview on spiking neural network learning approaches and open problems in this context can be found in Günnig and Bohte (2014).

7 Conclusion

In this work timed automata networks are proposed as an effective tool to model spiking neural networks. More precisely, we focus on LI&F networks. The model we describe has been fully implemented in Uppaal and is available at the pages (Ciatto et al. 2016; De Maria et al. 2017b). In our modeling framework, we take into account exact spike emission times of neurons rather than spike rates. The formalism we selected turned out to be very suited to model spiking neural networks: as a matter of fact, timed automata allow to model with precision several time-related aspects, such as the exact spike emission occurrences and the refractory period, a time interval which follows the spike emission and is characterised by a restricted emission capability.

In this work, model checking techniques are used to automatically prove our mapping of LI&F networks into timed automata is able to display a certain number of expected behaviours (i.e., typical responses to an input pattern), that is, tonic spiking, excitability, and integrator.

Formal verification turned out to be very suited to validate our modelling framework. As for future work concerning the modeling aspects, we intend to propose automaton-based formalisations for more sophisticated spiking neuron models, such as the theta-neuron model (Ermentrout and Kopell 1986) or the Izhikevich model (Izhikevich 2003). We also plan to enrich our model with propagation delays, which seem to play an important role in spiking neural networks (Paugam-Moisy and Bohte 2012). At this aim, we intend to add new states and clocks to model synapses. Finally, we intend to identify the neuron parameters which influence most the satisfaction of some crucial temporal logic properties. For this, a robustness analysis of the obtained model will be performed.

As key contribution, we proposed a novel technique to infer the synaptical weights of spiking neural networks. At this aim, we adapted machine learning techniques to bio-inspired models, which makes our work original and complementary with respect to the main international projects aiming at understanding the human brain, such as the Human Brain Project (D'Angelo et al. 2015), which mainly relies on large-scale simulations.

For our learning approach, we considered a basic kind of supervisors: each supervisor only focusses on the output of a single neuron, neglecting the other neurons. However, observe that the algorithm we propose still holds when supervisors compare the output of several neurons. As for future work, we plan to encode more complex supervisors which consider the behaviour of groups of neurons. Additionally, to enhance our learning technique, we could adapt some interesting results coming from the gene regulatory network area, where some necessary and sufficient conditions linking the structure of the network and its dynamics are given (Richard 2010).

To conclude, we program to generalise the ABP algorithm to infer some new parameters of neural networks, such as the leak factor or the firing threshold.

Acknowledgements We are grateful to Giovanni Ciatto for his preliminary implementation work and for his enthusiasm in collaborating with us.

References

- Ackley DH, Hinton GE, Sejnowski TJ (1988) A learning algorithm for Boltzmann machines. In: Waltz D, Feldman JA (eds) Connectionist models and their implications: readings from cognitive science. Ablex Publishing Corp., Norwood, pp 285–307
- Alur R, Dill DL (1994) A theory of timed automata. Theor Comput Sci 126(2):183–235
- Aman B, Ciobanu G (2016) Modelling and verification of weighted spiking neural systems. Theor Comput Sci 623:92–102. <https://doi.org/10.1016/j.tcs.2015.11.005>

- Bengtsson J, Larsen KG, Larsson F, Pettersson P, Yi W (1995) Uppaal—a tool suite for automatic verification of real-time systems. In: Proceedings of workshop on verification and control of hybrid systems III, no. 1066 in Lecture notes in computer science. Springer, pp 232–243
- Bohte SM, Poutr HAL, Kok JN, La HA, Joost P, Kok N (2002) Error-backpropagation in temporally encoded networks of spiking neurons. *Neurocomputing* 48:17–37
- Ciatto G, De Maria E, Di Giusto C (2016) Additional material. https://github.com/gciatto/snn_as_ta
- Cinquin O, Demongeot J (2005) High-dimensional switches and the modelling of cellular differentiation. *J Theor Biol* 233(3):391–411
- Clarke EM Jr, Grumberg O, Peled DA (1999) Model checking. MIT Press, Cambridge
- Cybenko G (1989) Approximation by superpositions of a sigmoidal function. *Math Control Signals Syst* 2(4):303–314
- D'Angelo E, Danese G, Florimbi G, Leporati F, Majani A, Masoli S, Solinas S, Torti E (2015) The human brain project: high performance computing for brain cells hw/sw simulation and understanding. In: 2015 Euromicro conference on digital system design, DSD 2015, Madeira, Portugal, August 26–28, 2015. IEEE Computer Society, pp 740–747. <https://doi.org/10.1109/DSD.2015.80>
- De Maria E, Di Giusto C (2018) Parameter learning for spiking neural networks modelled as timed automata. In: Anderson P, Gamboa H, Fred ALN, Badia SB (eds) Proceedings of the 11th international joint conference on biomedical engineering systems and technologies (BIOS-TEC 2018)—volume 3: BIOINFORMATICS, Funchal, January 19–21, 2018. SciTePress, pp 17–28. <https://doi.org/10.5220/0006530300170028>
- De Maria E, Muzy A, Gaffé D, Ressouche A, Grammont F (2016) Verification of temporal properties of neuronal archetypes using synchronous models. In: 5th International workshop on hybrid systems biology, Grenoble
- De Maria E, Di Giusto C, Ciatto G (2017a) Formal validation of neural networks as timed automata. In: Proceedings of the 8th international conference on computational systems-biology and bioinformatics, Nha Trang City, Viet Nam, December 7–8, 2017. ACM, pp 15–22. <https://doi.org/10.1145/3156346.3156350>
- De Maria E, Di Giusto C, Laversa L (2017b) Additional material. <https://digiusto.bitbucket.io/>
- Ermentrout GB, Kopell N (1986) Parabolic bursting in an excitable system coupled with a slow oscillation. *SIAM J Appl Math* 46(2):233–253
- Freund Y, Schapire RE (1999) Large margin classification using the perceptron algorithm. *Mach Learn* 37(3):277–296
- Gerstner W, Kistler W (2002) Spiking neuron models: an introduction. Cambridge University Press, New York
- Gruber H, Holzer M, Kiehn A, König B (2005) On timed automata with discrete time-structural and language theoretical characterization. In: Developments in language theory, 9th international conference, DLT 2005, Palermo, Italy, July 4–8, 2005, Proceedings, pp 272–283. <https://doi.org/10.1007/1150587724>
- Gürning A, Bohte S (2014) Spiking neural networks: principles and challenges. In: 22nd European symposium on artificial neural networks, ESANN 2014, Bruges, Belgium, April 23–25, 2014. <http://www.elen.ucl.ac.be/Proceedings/esann/esannpdf/es2014-13.pdf>
- Hebb DO (1949) The organization of behavior. Wiley, Hoboken
- Hodgkin AL, Huxley AF (1952) A quantitative description of membrane current and its application to conduction and excitation in nerve. *J Physiol* 117(4):500–544
- Hopfield JJ (1988) Neural networks and physical systems with emergent collective computational abilities. In: Anderson JA, Rosenfeld E (eds) Neurocomputing: foundations of research. MIT Press, Cambridge, pp 457–464
- Ijspeert AJ (2008) Central pattern generators for locomotion control in animals and robots: a review. *Neural Netw* 21(4):642–653. <https://doi.org/10.1016/j.neunet.2008.03.014>
- Izhikevich EM (2003) Simple model of spiking neurons. *Trans Neural Netw* 14(6):1569–1572
- Izhikevich EM (2004) Which model to use for cortical spiking neurons? *IEEE Trans Neural Netw* 15(5):1063–1070
- Lapicque L (1907) Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation. *J Physiol Pathol Gen* 9:620–635
- Maass W (1997) Networks of spiking neurons: the third generation of neural network models. *Neural Netw* 10(9):1659–1671
- Matsuoka K (1987) Mechanisms of frequency and pattern control in the neural rhythm generators. *Biol Cybern* 56(5–6):345–353
- McCulloch WS, Pitts W (1943) A logical calculus of the ideas immanent in nervous activity. *Bull Math Biophys* 5(4):115–133
- Paugam-Moisy H, Bohte S (2012) Computing with spiking neuron networks. Springer, Berlin, pp 335–376
- Recce M (1999) Encoding information in neuronal activity. In: Maass W, Bishop CM (eds) Pulsed neural networks. MIT Press, Cambridge, pp 111–131
- Richard A (2010) Negative circuits and sustained oscillations in asynchronous automata networks. *Adv Appl Math* 44(4):378–392
- Rumelhart DE, Hinton GE, Williams RJ (1988) Learning representations by back-propagating errors. In: Anderson JA, Rosenfeld E (eds) Neurocomputing: foundations of research. MIT Press, Cambridge, pp 696–699
- Sjostrom J, Gerstner W (2010) Spike-timing dependent plasticity. *Scholarpedia* 5(2):1362

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.